

Contrôle continu - 2 exercices

Mars 2008

Rendez les deux exercices sur des copies séparées.

Exercice 1 – Horloges et Causalité

On rappelle la propriété CO : $\forall m, m'$ deux messages, $send(m) \rightarrow send(m') \Rightarrow rec(m) \rightarrow rec(m')$

Question 1

Peut-on utiliser les horloges logiques pour déterminer si une exécution vérifie la propriété CO ? Justifiez précisément votre réponse.

Solution:

Pour déterminer si une exécution vérifie la propriété CO, il faut pouvoir reconstruire la relation de causalité entre les émissions de messages, et entre les réceptions.

Ce n'est pas possible avec les horloges scalaires puisque $H(e) < H(e')$ ne permet pas de savoir si $e \rightarrow e'$. Par contre les horloges vectorielles vérifient $VC(e) < VC(e') \Leftrightarrow e \rightarrow e'$, donc on pourrait écrire de manière équivalente la propriété CO par $VC(send(m)) < VC(send(m')) \Rightarrow VC(rec(m)) < VC(rec(m'))$.

Fin solution

Question 2

On a lancé l'exécution d'une application répartie sur 3 sites et relevé les valeurs d'horloges vectorielles des différents événements exécutés.

Sur le site 1 : $\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 3 \\ 1 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 4 \\ 1 \\ 0 \end{bmatrix}$. Sur le site 2 : $\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 2 \\ 3 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 2 \\ 4 \\ 2 \end{bmatrix}$.

Sur le site 3 : $\begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$, $\begin{bmatrix} 2 \\ 3 \\ 2 \end{bmatrix}$, $\begin{bmatrix} 4 \\ 3 \\ 3 \end{bmatrix}$.

Représentez l'exécution correspondante. Citez deux événements concurrents.

Cette exécution vérifie-t-elle la propriété CO ?

Solution:

Voir figure 1.

Les événements e_1^3 et e_3^2 ont des vecteurs d'horloge non comparables, ils sont donc concurrents.

$send(m) \rightarrow send(m')$ mais $rec(m) || rec(m')$: l'exécution ne vérifie donc pas la propriété CO.

Fin solution

Question 3

Complétez l'exécution de l'algorithme de Singhal et Kshemkalyani pour l'exemple de la figure 2. Vous représenterez l'évolution des vecteurs d'horloge ainsi que l'information véhiculée par les messages, en expliquant la manière dont elle est obtenue.

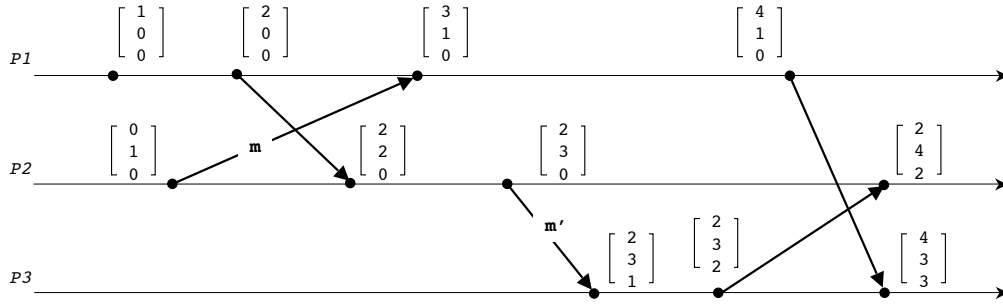


FIG. 1 – Construction d'une exécution répartie

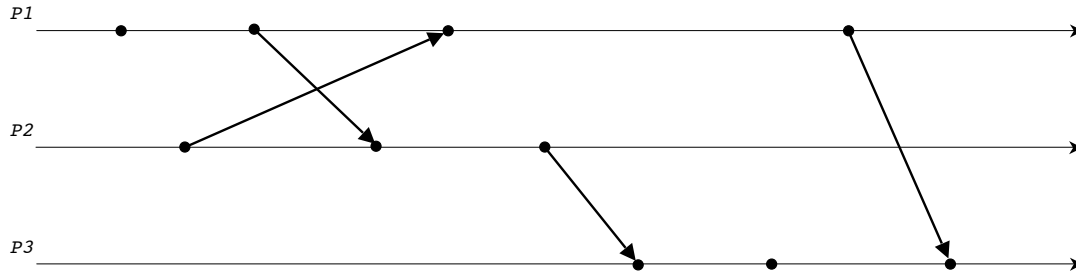


FIG. 2 – Exécution de l'algorithme de Singhal et Kshemkalyani

Pour cela, vous préciserez les valeurs initiales des vecteurs LU et LS utilisés par l'algorithme ainsi que les valeurs après l'exécution d'un événement e_i^j lorsque celui-ci modifie un vecteur.

Solution:

Voir figure 3. Tous les vecteurs sont initialisés à 0. On traite les événements par ordre chronologique.

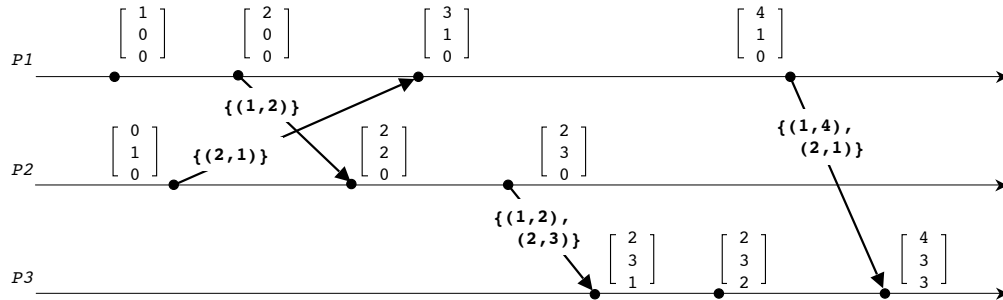


FIG. 3 – Exécution de l'algorithme S-K

$$e_1^1 : LU_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}. e_2^1 : LU_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}. LU_2[2] > LS_2[1] \Rightarrow \text{émission de } (2, 1). \text{ Après émission, } LS_2 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}.$$

$$e_1^2 : LU_1 = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}. LU_1[1] > LS_1[2] \Rightarrow \text{émission de } (1, 2). \text{ Après émission, } LS_1 = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix}.$$

$$e_2^2 : LU_2 = \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix}. e_1^3 : LU_1 = \begin{bmatrix} 3 \\ 3 \\ 0 \end{bmatrix}.$$

$$e_2^3 : LU_2 = \begin{bmatrix} 2 \\ 3 \\ 0 \end{bmatrix}. LU_2[1] > LS_2[3] \text{ et } LU_2[2] > LS_2[3] \Rightarrow \text{émission de } \{(1, 2), (2, 3)\}. \text{ Après, } LS_2 = \begin{bmatrix} 1 \\ 0 \\ 3 \end{bmatrix}.$$

$$e_3^1 : LU_3 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}. e_3^2 : LU_3 = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}.$$

$$e_1^4 : LU_1 = \begin{bmatrix} 4 \\ 3 \\ 0 \end{bmatrix}. LU_1[1] > LS_1[3] \text{ et } LU_1[2] > LS_1[3] \Rightarrow \text{émission de } \{(1, 4), (2, 1)\}. \text{ Après, } LS_1 = \begin{bmatrix} 0 \\ 2 \\ 4 \end{bmatrix}.$$

$$e_3^3 : LU_3 = \begin{bmatrix} 3 \\ 1 \\ 3 \end{bmatrix}.$$

Fin solution

Exercice 2 – Exclusion mutuelle

Nous considérons une topologie maillée avec N sites : chaque site peut communiquer avec tous les autres.

Un jeton $jeton_{i,j}$ est associé à toute paire de sites (S_i, S_j) , $S_i \neq S_j$. À tout instant, soit le site S_i possède le jeton $jeton_{i,j}$, soit c'est le site S_j . Si le site S_i possède le jeton $jeton_{i,j}$, il le garde tant qu'il est en section critique, et tant que S_j ne le demande pas.

Cependant, lorsque S_j demande le jeton à S_i , vous devrez fixer les critères (règles) qui déterminent si S_i doit ou non céder le jeton, afin de garantir la propriété de *vivacité* (voir question 4).

Pour pouvoir entrer en section critique, le site S_i doit collecter tous les jetons $jeton_{i,k}$, $1 \leq k \leq N$ (et $k \neq i$). Il doit donc demander ceux qui lui manquent. Lorsqu'il possède ces $(N - 1)$ jetons, le site S_i peut entrer en section critique (propriété de *sûreté*).

En sortant de la section critique, S_i renvoie aux sites correspondants tous les $jeton_{i,k}$ qui ont été demandés ($1 \leq k \leq N$, $k \neq i$ et k a demandé le jeton $jeton_{i,k}$). S_i garde pour lui les autres jetons.

Observations :

- $jeton_{i,j}$ et $jeton_{j,i}$ désignent le même jeton ;
- l'algorithme n'utilise aucun autre jeton que ceux que l'on a définis.

Question 1

En considérant N sites, combien de jetons y a-t-il dans le système ?

Solution:

$$C_{N,2} = (N * N - 1)/2$$

Fin solution

Question 2

Expliquez de façon informelle pourquoi la propriété de sûreté est assurée.

Solution:

Si le site S_i est en section critique, il possède tous les jetons $jeton_{i,k}$ $1 \leq k \leq N$ et $k \neq i$. Si un deuxième site S_j veut lui aussi entrer en section critique, il a besoin des $(N - 1)$ jetons $jeton_{j,k}$, y compris le jeton $jeton_{i,j}$ que i possède et qu'il conservera jusqu'à ce qu'il sorte de la section critique. Donc le site S_j ne peut pas entrer en section critique.

Fin solution

Question 3

Montrez sur un exemple simple (3 sites) que si un site demandeur ne cède jamais un jeton qu'il possède jusqu'à ce qu'il ait exécuté sa section critique, on peut créer un problème de famine.

Solution:

Supposons que le site 1 possède le jeton12, le site 2 possède le jeton 23 et le site 3 possède le jeton13. Si tous les 3 demandent la SC en même temps, aucun ne cède son jeton et on arrive à un interblocage (donc famine).

Fin solution

Question 4

Proposez une règle à appliquer pour déterminer si un site S_i doit transmettre le jeton $jeton_{i,j}$ au site S_j lorsque celui-ci le demande, afin de garantir la propriété de *vivacité* de l'algorithme.

Solution:

On définit un ordre total pour les messages de requête : dater les messages *REQUEST* avec l'horloge de Lamport + identifiant. La propriété de vivacité est alors assurée.

Fin solution

Question 5

Programmez les fonctions *Request_CSi()* et *Release_CSi()* appelées par le processus S_i respectivement pour demander l'accès à la section critique et pour la libérer.

Programmez les fonctions *Reception_Requete_i(j, ..)* et *Reception_Jeton_i(j)* appelées par S_i respectivement lors de la réception d'une requête émise par S_j , et lors de la réception du jeton *jeton_{i,j}* envoyé par S_j .

Observations :

- Vous pouvez ajouter d'autres paramètres à la fonction *Reception_Requete_i(j, ...)* ainsi que des champs dans les messages, si nécessaire, toujours en les justifiant ;
- L'initialisation des variables doit être spécifiée ;
- Votre solution doit envoyer juste le nombre de messages nécessaire.

Solution:

```

set Token_Request_set    /*sites a qui i doit demander le jeton */
set Defer_set;
int state;
int last, Hi;

for (j=1; j <= N and i>j; j++)
    Token_Request_set = Token_Request_set U {j};
    /* si i < j, i possede jeton_ij a l'initialisation */

/* Message REQUEST : <REQUEST, (Hi,i)> : i identifiant du site demandeur. */
/* Message JETON ; <JETON,i>

Request_CSi ( ) {
    state = requesting;
    Hi++;
    last = Hi;

    for (k=1; k <= N and k in Token_Request_set; k++)
        send (REQUEST, (i, last)) to k;
    wait (Token_Request_set = {});
    state= CS;
}

Release_CSi ( ) {
    for (k=1; k <= N and k in Defer_set; k++)
        send (TOKEN,i) to k;
    Token_Request_set = Defer_set;
    Defer_set = {};
    state = CS;
}

Reception_requete (j, Hj) {
    Hi = max(Hi,Hj);
    if ( (state = CS) ou ((state = requesting) and (last<Hj)))
        Defer_set = Defer_Set U {j};
    else {
        Token_Request_set =Token_Request_set U {j};
    }
}

```

```
    send (TOKEN,i)) to k;
    if (state = requesting)
        send (REQUEST, (i, last)) to j;
}
}
```



```
Reception_jeton(j) {
    Token_Request_set =Token_Request_set - {j};
}
```

Fin solution**Question 6**

Quel est le nombre de messages envoyés par section critique ? Justifiez votre réponse.

Solution:

Le nombre varie entre 0 et $2(N-1)$ messages par section critique en fonction des demandes des sites et de l'état du système.

- 0 : le site possède déjà les $(N - 1)$ jetons nécessaires ;
- $2(N - 1)$: le site ne possède aucun des jetons dont il a besoin. Donc $(N - 1)$ demandes et $(N - 1)$ réceptions.

Fin solution**Question 7**

Quels sont les avantages et/ou inconvénients de cet algorithme par rapport à l'algorithme Ricart-Agrawala ? Et par rapport à l'algorithme de Suzuki-Kasami ?

Solution:

Avantage par rapport à Agrawala : l'algorithme envoie moins de messages. Inconvénient par rapport à Suzuki : celui-ci ne gère qu'un jeton et le nombre de message par section critique est 0 ou N.

Fin solution